

Tugas 1

Nama: Adelia Isra Ekaputri

NIM: 10241004

Kode

```
<?php
abstract class MenuItem {
    protected string $name;
    protected float $price;

    public function __construct(string $name, float $price) {
        if (empty($name)) {
            throw new Exception("Nama menu tidak boleh kosong!");
        }
        if ($price < 0) {
            throw new Exception("Harga tidak boleh negatif!");
        }
        $this->name = $name;
        $this->price = $price;
    }

    public function getName(): string {
        return $this->name;
    }

    public function getPrice(): float {
        return $this->price;
    }

    abstract public function cook(): string;
}

class FriedMenu extends MenuItem {
    public function cook(): string {
        return "Menggoreng " . $this->name;
    }
}

class GrilledMenu extends MenuItem {
    public function cook(): string {
        return "Membakar " . $this->name;
    }
}

class BoiledMenu extends MenuItem {
    public function cook(): string {
        return "Merebus " . $this->name;
    }
}
```

```
    }  
}  
  
class Order {  
    private array $items = [];  
  
    public function addItem(MenuItem $item) {  
        $this->items[] = $item;  
    }  
  
    public function getItems(): array {  
        return $this->items;  
    }  
  
    public function calculateTotal(): float {  
        $total = 0;  
        foreach ($this->items as $item) {  
            $total += $item->getPrice();  
        }  
        return $total;  
    }  
}  
  
interface PaymentMethod {  
    public function pay(float $amount): string;  
}  
  
class CashPayment implements PaymentMethod {  
    public function pay(float $amount): string {  
        return "Dibayar tunai sebesar Rp " . number_format($amount, 0, ',', '.');  
    }  
}  
  
class CreditCardPayment implements PaymentMethod {  
    public function pay(float $amount): string {  
        return "Dibayar dengan kartu kredit sebesar Rp " . number_format($amount,  
0, ',', '.');  
    }  
}  
  
class QRISPayment implements PaymentMethod {  
    public function pay(float $amount): string {  
        return "Dibayar melalui QRIS sebesar Rp " . number_format($amount, 0, ',',  
'.'');  
    }  
}  
  
try {  
    $order = new Order();  
  
    $menu1 = new FriedMenu("Ayam Goreng", 25000);  
    $menu2 = new GrilledMenu("Ikan Bakar", 40000);  
    $menu3 = new BoiledMenu("Sayur Sop", 15000);
```

```

$order->addItem($menu1);
$order->addItem($menu2);
$order->addItem($menu3);

echo "==== STRUK PEMESANAN ====<br>";
foreach ($order->getItems() as $item) {
    echo "- " . $item->getName() .
        " | Rp " . number_format($item->getPrice(), 0, ',', '.') .
        " | " . $item->cook() . "<br>";
}

$total = $order->calculateTotal();
echo "-----<br>";
echo "TOTAL: Rp " . number_format($total, 0, ',', '.') . "<br>";

$payment = new QRISPayment();
echo $payment->pay($total) . "<br>";

} catch (Exception $e) {
    echo "Error: " . $e->getMessage();
}
?>

```

Penjelasan Kode

1. Abstract Class `MenuItem`

```

abstract class MenuItem {
    protected string $name;
    protected float $price;
    ...
    abstract public function cook(): string;
}

```

◦ Fungsi:

- `name` - nama menu (tidak boleh kosong).
- `price` - harga menu (tidak boleh negatif).
- `cook()` - method abstrak (harus diimplementasikan oleh class turunan).

Konsep OOP: Abstraction, membuat class dasar yang tidak bisa dipakai langsung, tapi jadi blueprint untuk class lain.

2. Inheritance (pewarisan)

```

class FriedMenu extends MenuItem {
    public function cook(): string {
        return "Menggoreng " . $this->name;
    }
}

```

- `FriedMenu`, `GrilledMenu`, `BoiledMenu` - semuanya turunan dari `MenuItem`.
- Masing-masing mendefinisikan cara masakannya sendiri:
 - `FriedMenu` - Menggoreng
 - `GrilledMenu` - Membakar
 - `BoiledMenu` - Merebus

Konsep OOP: Inheritance, class anak mewarisi atribut (`name`, `price`) dari class induk.

3. Encapsulation (pengendalian data)

```
class Order {  
    private array $items = [];  
  
    public function addItem(MenuItem $item) { ... }  
    public function getItems(): array { ... }  
    public function calculateTotal(): float { ... }  
}
```

- `items` dibuat private, agar tidak bisa diakses langsung dari luar.
- Untuk menambah menu, harus lewat `addItem()`.
- Untuk menghitung total harga, ada method `calculateTotal()`.

Konsep OOP: Encapsulation, data disembunyikan, akses hanya lewat method resmi.

4. Polymorphism (berbagai cara pembayaran)

```
interface PaymentMethod {  
    public function pay(float $amount): string;  
}
```

- Ada interface `PaymentMethod` - semua metode pembayaran wajib punya `pay()`.
- Implementasi berbeda-beda:
 - `CashPayment` - bayar tunai.
 - `CreditCardPayment` - bayar kartu kredit.
 - `QRISPayment` - bayar via QRIS.

Konsep OOP: Polymorphism, satu interface, tapi banyak bentuk implementasi.

5. Simulasi pemesanan

```
$order = new Order();  
$menu1 = new FriedMenu("Ayam Goreng", 25000);  
$menu2 = new GrilledMenu("Ikan Bakar", 40000);  
$menu3 = new BoiledMenu("Sayur Sop", 15000);
```

```
$order->addItem($menu1);  
$order->addItem($menu2);  
$order->addItem($menu3);
```

- Membuat pesanan dengan 3 menu.
- Menyimpan pesanan ke dalam **Order**.
- Menghitung total, lalu menampilkan struk.

Penjelasan Output

Jika program dijalankan di browser, maka hasilnya adalah:

```
===== STRUK PEMESANAN =====  
- Ayam Goreng | Rp 25.000 | Menggoreng Ayam Goreng  
- Ikan Bakar | Rp 40.000 | Membakar Ikan Bakar  
- Sayur Sop | Rp 15.000 | Merebus Sayur Sop  
-----  
TOTAL: Rp 80.000  
Dibayar melalui QRIS sebesar Rp 80.000
```

Berikut untuk makna-makna dari output tersebut:

1. Header

===== STRUK PEMESANAN ===== , yaitu untuk menandakan awal dari struk tersebut.

2. Detail Pesanan

```
- Ayam Goreng | Rp 25.000 | Menggoreng Ayam Goreng  
- Ikan Bakar | Rp 40.000 | Membakar Ikan Bakar  
- Sayur Sop | Rp 15.000 | Merebus Sayur Sop
```

Bagian ini menampilkan nama menu, harga, dan cara memasaknya (sesuai dengan polimorfisme di class **cook()**).

3. Total Harga

```
-----  
TOTAL: Rp 80.000
```

Bagian ini menampilkan hasil dari **calculateTotal()**, yaitu jumlah dari semua harga menu yang dipesan.

4. Pembayaran

Dibayar melalui QRIS sebesar Rp 80.000

Hasil dari method `pay()` di class `QRISPayment`. Apabila diganti `CashPayment` atau `CreditCardPayment`, maka output nya pada bagian ini juga akan berubah.